Name: Rhilo Sotto
RED ID: 130551574
Date: 9/13/2024

Lab 1 Report

1. What does your program do?

The program outputs an arbitrary string by blinking the LED in Morse code, dots being 1 unit, dashes being 3 units, space between the same letter being 1 unit, space between letters being 3 units, and space between words being 7 units; with units being 200ms long. The program runs continuously in an infinite loop, and since there is no break statement, it will run indefinitely until power is lost.

2. What variables/registers does your program use?

The program uses the DDRB and PORTB registers. DDRB sets the direction of data for individual pins, in this case PORTB's 5th pin, outputting to the on-board LED.
PORTB controls the value outputted to those pins, in this case focusing on the 5th bit of this register that is connected to the on-board LED.

The morseCode function uses the unsigned 8 bit variable, stringIndex, to iterate through each character of the inputted string.

3. What is the main algorithm/logic of your program?

The main program includes one line of code that sets the data direction of the 5th pin of PORT B as output, outside of the infinite while loop. After that point, the infinite while loop starts.

In the loop body, the morseCode function is called with a string as the parameter. This will iterate through the string starting at the 0 index and terminate at the null character ('\0') at the end of the string using stringIndex. Using a switch statement with the character at stringIndex of the input string, we determine which ASCII character we need to blink in morse code, which is the morse code table hard-coded into dots and dashes for each character.

The dot and dash functions are called to turn on the LED for a period of time (1 or 3 units respectively) and then turn off the LED and keep it off for a period of time (1 unit).
wordSpace is called for ' ' characters, which is in the space between words and numbers in the string. After the switch statement, letterSpace is called to keep the LED off for the space between letters.

After the morseCode function is finished, wordSpace is called as the rereading the string should be considered the start of a new word.

The space functions (letterSpace, wordSpace) do not directly use the corresponding units (3, 7) in the function definition as it is implied that they will be called following either a dot or dash function and/or proceeding a letterSpace. The total space by calling dot/dash,

wordSpace (in the case of ‘ ‘, the gap between words or the end of string), and letterSpace will add up to the correct units of space required.

4. What external hardware connections did you make?

There are no external hardware connections made since the on-board LED is used for output.

Appendix (Code)

```
/*
 * Lab1.c
 *
 * Created: 9/5/2024 2:25:19 PM
 * Author : rnsot
 */

// 1 unit is 200ms
#define F_CPU 16000000UL  // 16MHz clock from the debug processor
#include <avr/io.h>
#include <util/delay.h>

void dot(void) {
	PORTB |= (1<<PORTB5); // light on
	_delay_ms(200); // 1 unit
	PORTB &= ~(1<<PORTB5); // light off
	_delay_ms(200); // 1 unit

}

void dash(void) {
	PORTB |= (1<<PORTB5); // light on
	_delay_ms(600); // 3 units
	PORTB &= ~(1<<PORTB5); // light off
	_delay_ms(200); // 1 unit

}

void letterSpace(void) {
	PORTB &= ~(1<<PORTB5); // ensure light is off
```

```
	_delay_ms(400); // 3 units (2 units + 1 unit is implicit with dot/dash function)
}

void wordSpace(void) {
	PORTB &= ~(1<<PORTB5); // ensure light is off
	_delay_ms(800); // 7 units (4 units + 2 units is implicit since letterSpace() is always
called + 1 unit is implicit with dot/dash function)
}

void morseCode(char* string) {
	uint8_t stringIndex = 0;
	while(string[stringIndex] != '\0') { // until the end of string is reached
		switch (string[stringIndex]) {
			// letters
			case 'A':
			case 'a':
				dot(); dash();
				break;
			case 'B':
			case 'b':
				dash(); dot(); dot(); dot();
				break;
			case 'C':
			case 'c':
				dash(); dot(); dash(); dot();
				break;
			case 'D':
			case 'd':
				dash(); dot(); dot();
				break;
			case 'E':
			case 'e':
				dot();
				break;
			case 'F':
			case 'f':
				dot(); dot(); dash(); dot();
				break;
			case 'G':
			case 'g':
				dash(); dash(); dot();
				break;
			case 'H':
			case 'h':
```

```
            dot(); dot(); dot(); dot();
            break;
case 'I':
case 'i':
            dot(); dot();
            break;
case 'J':
case 'j':
            dot(); dash(); dash(); dash();
            break;
case 'K':
case 'k':
            dash(); dot(); dash();
            break;
case 'L':
case 'l':
            dot(); dash(); dot(); dot();
            break;
case 'M':
case 'm':
            dash(); dash();
            break;
case 'N':
case 'n':
            dash(); dot();
            break;
case 'O':
case 'o':
            dash(); dash(); dash();
            break;
case 'P':
case 'p':
            dot(); dash(); dash(); dot();
            break;
case 'Q':
case 'q':
            dash(); dash(); dot(); dash();
            break;
case 'R':
case 'r':
            dot(); dash(); dot();
            break;
case 'S':
case 's':
```

```
            dot(); dot(); dot();
            break;
case 'T':
case 't':
            dash();
            break;
case 'U':
case 'u':
            dot(); dot(); dash();
            break;
case 'V':
case 'v':
            dot(); dot(); dot(); dash();
            break;
case 'W':
case 'w':
            dot(); dash(); dash();
            break;
case 'X':
case 'x':
            dash(); dot(); dot(); dash();
            break;
case 'Y':
case 'y':
            dash(); dot(); dash(); dash();;
            break;
case 'Z':
case 'z':
            dash(); dash(); dot(); dot();
            break;
// numbers
case '1':
            dot(); dash(); dash(); dash(); dash();
            break;
case '2':
            dot(); dot(); dash(); dash(); dash();
            break;
case '3':
            dot(); dot(); dot(); dash(); dash();
            break;
case '4':
            dot(); dot(); dot(); dot(); dash();
            break;
case '5':
```

```
                        dot(); dot(); dot(); dot(); dot();
                        break;
                case '6':
                        dash(); dot(); dot(); dot(); dot();
                        break;
                case '7':
                        dash(); dash(); dot(); dot(); dot();
                        break;
                case '8':
                        dash(); dash(); dash(); dot(); dot();
                        break;
                case '9':
                        dash(); dash(); dash(); dash(); dot();
                        break;
                case '0':
                        dash(); dash(); dash(); dash(); dash();
                        break;
                // space between words and beginning of numbers
                case ' ':
                        wordSpace();
                        break;
                // fail case
                default:
                        break;
            }
            letterSpace(); // always going to call letter space
            // delay of dot/dash = space between part of same letter
            // delay of dot/dash + letterSpace = space between letters
            // delay of dot/dash + wordSpace + letterSpace = space between words
            ++stringIndex; // next character
        }
}

int main(void)
{
        DDRB |= (1<<DDB5);  //0x20 (hex) // Set port bit B5 in data direction register to 1: an
OUTput
        // TESTING ALL MORSE CODE
        // "A a B b C c D d E e F f G g H h I i J j K k L l M m N n O o P p Q q R r S s T t U u V v
W w X x Y y Z z 1 2 3 4 5 6 7 8 9 0";

        // null terminated ASCII string
        // breaking up words and number with ASCII space character
        while(1) {
```

```
            morseCode("Rhilo Sotto 130551574");
            wordSpace(); // end of the loop means a new word
            letterSpace(); // gap between words is dot/dash(1) + word(4) + letter(2) = (7)
        }
        // "Rhilo Sotto 130551574"
        /*
        R - dot dash dot
        H - dot dot dot dot
        I - dot dot
        L - dot dash dot dot
        O - dash dash dash

        S - dot dot dot
        O - dash dash dash
        T - dash
        T - dash
        O - dash dash dash

        1 - dot dash dash dash dash
        3 - dot dot dot dash dash
        0 - dash dash dash dash dash
        5 - dot dot dot dot dot
        5 - dot dot dot dot dot
        1 - dot dash dash dash dash
        5 - dot dot dot dot dot
        7 - dash dash dot dot dot
        4 - dot dot dot dot dash
        */
}
```